Active Learning for Genomic Sequence Classification with DNABERT-2

Jingyao Chen, Joe Wang

June 30, 2026

Abstract

We study pool-based active learning for genomic sequence classification with DNABERT-2 across promoter detection, enhancer detection, and a 20-class fungi benchmark. The report focuses on four questions: whether active learning improves label efficiency over random sampling, whether cold-start initialisation matters, whether transfer pretraining changes downstream acquisition behaviour, and when D-Optimal design is useful in practice. Across the main benchmarks, BADGE is the most reliable acquisition function, while TypiClust helps only at the very smallest label budget and D-Optimal is effective only when the embedding space is both frozen and well aligned with the target task. Transfer from mouse to human promoters yields a large, persistent gain. Under a fixed total labelling budget, query batch size has a secondary effect compared with the choice of acquisition strategy.

1 Introduction

Foundation models for DNA sequence analysis make it feasible to train strong genomic classifiers from relatively small labelled datasets, but obtaining those labels remains expensive. In this setting, active learning (AL) is attractive because it aims to spend a limited annotation budget on the most informative sequences rather than sampling uniformly at random.

This report evaluates AL with DNABERT-2 on a set of genomic sequence classification problems with different levels of difficulty and domain shift: mouse promoter detection, human promoter transfer, NT enhancer detection, and a 20-class fungi benchmark. Beyond a simple benchmark of acquisition functions, we study three additional practical questions that arise in low-budget genomic annotation workflows: whether the initial seed set matters, whether transfer pretraining helps, and whether classical D-Optimal design remains useful when the feature space comes from a deep model.

Our analysis leads to three high-level conclusions. First, diversity- aware acquisition is generally more reliable than pure uncertainty sampling on these tasks. Second, cold-start and batch-size choices are usually secondary to the acquisition rule itself. Third, transfer and D-Optimal querying are highly context dependent: both work well when the representation geometry is stable and task aligned, but both can fail when representation geometry is stable and task aligned.

2 Background

Pool-based active learning. In pool-based AL, a learner iteratively selects batches of unlabelled examples to annotate from a fixed candidate pool. At each round the model is retrained on the accumulated labelled set, and the acquisition function uses the current model to score unlabelled candidates. The goal is to reach a target performance level with fewer labels than uniform random sampling. Acquisition functions span three broad families: *uncertainty-based* methods (e.g. BALD, Entropy Sampling) select examples where the model is most uncertain; *diversity-based* methods (e.g. CoreSet) maximise coverage of the embedding space; and *hybrid* methods (e.g. BADGE) combine both signals [1, 7, 4].

DNA foundation models. DNABERT [5] introduced the BERT pre-training paradigm to genomic sequences, demonstrating that transformer representations transfer across regulatory element prediction tasks. DNABERT-2 [8] extends this with multi-species pre-training and a Byte-Pair Encoding

tokeniser, substantially improving transfer and label efficiency. These representations are central to the AL setting studied here: because the backbone already encodes general genomic structure, even small labelled sets can yield useful classifiers, and the geometry of the embedding space is informative enough to guide acquisition.

Active learning for genomics. Most prior work on genomic sequence classification assumes a fully supervised setting with large labelled datasets. Active learning has been applied to protein fitness prediction and variant effect modelling, but its systematic evaluation on regulatory element benchmarks with modern foundation models is limited. This report fills part of that gap by benchmarking a representative set of acquisition strategies on tasks where annotation cost is a practical concern.

3 Methods and Data

3.1 Model

All experiments use DNABERT-2 [8] (117M parameters), a BERT-style encoder pretrained on the multi-species genome with a Byte-Pair Encoding tokeniser designed for DNA sequences. The model takes raw nucleotide sequences (up to 128 bp) as input and produces a fixed-dimensional `[CLS]` embedding. A two-layer classification head (linear projection $\rightarrow$ GELU $\rightarrow$ linear) is attached for downstream fine-tuning.

During active learning, the full model is fine-tuned from the current labeled set at each round using AdamW (lr $= 2 \times 10^{-5}$, weight decay 0.01, cosine schedule with 10% warmup, 5 epochs per round, batch size 16). In the transfer-then-freeze variant the backbone weights are fixed after pretraining and only the head is updated (lr $= 10^{-3}$, same schedule).

3.2 Datasets

Mouse promoter (mm). Binary classification: core promoter vs. non-promoter. Positive sequences (TATA and non-TATA promoters, 70 bp) are sourced from species-specific text files; negatives are generated by dinucleotide-preserving shuffling at a 1:1 ratio. After shuffling with seed 42, 80% of sequences are reserved for the active learning pool and 20% for evaluation.

Human promoter (hs). Same construction as mouse, using *Homo sapiens* promoter files. Used exclusively as the target domain in the mm$\rightarrow$hs transfer experiment. The evaluation set is held out identically to the mouse split.

NT Enhancers. Binary enhancer detection task from the Nucleotide Transformer benchmark was loaded via HuggingFace `datasets`. We use the provided train/test splits (27,100 training, 9,031 test sequences, 251 bp each).

Fungi (GUE). Twenty-class genomic sequence classification task from the GUE benchmark suite [8]. We use the benchmark split and train a 20-way classifier on top of DNABERT-2.

Table 1 summarises the datasets and their role in the study.

Table 1: Dataset summary.

Dataset	Task	Role in study	Split	Seq. length
Mouse Promoter (mm)	Binary	Main benchmark; transfer source	80/20 split	70 bp
Human Promoter (hs)	Binary	Transfer target	80/20 split	70 bp
NT Enhancers	Binary	Main benchmark; cold-start target	Provided train/test split	251 bp
Fungi (GUE)	20-class	Main benchmark	Benchmark split	Dataset-specific

3.3 Active Learning Protocol

All experiments follow a pool-based AL protocol. At round $t = 0$ a seed set of $n_{\text{init}} = 100$ sequences is drawn uniformly at random from the training pool unless a dedicated initialisation study is being run. At each subsequent round the query strategy selects a batch of n_{query} additional sequences to label until a maximum budget of $n = 2{,}000$ is reached. The default setting is $n_{\text{init}} = 100$ and $n_{\text{query}} = 100$ (20 rounds), but two experiment families deviate from this template: the fungi batch-size study varies $n_{\text{query}} \in \{20, 50, 200\}$ while holding the total budget fixed, and the DOE study additionally sweeps $n_{\text{init}} \in \{100, 300, 500\}$. Performance (macro-F1) is measured on the held-out test set after each round using the model trained on the cumulative labeled set. All experiments are repeated with 3 random seeds (42, 43, 44); results are reported as mean ± std.

3.4 Uncertainty Estimation via Monte Carlo Dropout

A recurring challenge in deep active learning is obtaining calibrated uncertainty estimates from a deterministic neural network. We follow the approach of Gal and Ghahramani [2], who showed that a network trained with dropout and evaluated with dropout *enabled at inference time* is equivalent to approximate Bayesian inference in a deep Gaussian process. Concretely, given T stochastic forward passes through the network (each with a different random dropout mask), the predictive distribution over class c is approximated as

$$p(y = c \mid x) \approx \frac{1}{T} \sum_{t=1}^{T} p(y = c \mid x, \hat{\theta}_t), \tag{1}$$

where $\hat{\theta}_t$ denotes the network weights under the t-th dropout mask. The mean over passes estimates the predictive probability, and the spread across passes captures model (epistemic) uncertainty. In our experiments we use $T = 10$ forward passes throughout. This mechanism underlies both the Entropy Sampling and BALD acquisition functions described below, as well as the Batch BALD approximation.

3.5 Acquisition Functions

We evaluate seven acquisition functions spanning the diversity, uncertainty, and information-theoretic families.

Random Sampling. Selects n sequences uniformly at random from the unlabeled pool. Serves as the primary baseline.

Entropy Sampling. Selects the n unlabeled sequences with the highest predictive entropy: $H[y|x] = -\sum_c p_c(x) \log p_c(x)$. Uses Monte Carlo Dropout [2] ($T = 10$ forward passes) to obtain a distribution over predictions.

CoreSet (k-Center Greedy) [7]. Selects sequences that maximise coverage of the embedding space. At each step the unlabeled point with the maximum minimum Euclidean distance to any currently labeled point is chosen:

$$x^* = \arg\max_{x \in \mathcal{U}} \min_{x' \in \mathcal{L}} \|f(x) - f(x')\|_2, \tag{2}$$

where $f(\cdot)$ denotes the `[CLS]` embedding from the current model. The greedy k-center procedure is repeated n times per round.

BADGE. Selects a diverse set of sequences in *gradient embedding* space. For each unlabeled sequence x with predicted class $\hat{y}$, the gradient embedding is

$$g_c(x) = f(x) \cdot \big(\mathbf{1}[c = \hat{y}] - p_c(x)\big), \tag{3}$$

capturing both uncertainty (via the softmax residual) and representation (via the embedding). A batch of n sequences is then selected by k-means++ initialisation on $\{g(x) : x \in \mathcal{U}\}$, promoting both diversity and uncertainty.

BALD [4]. Selects sequences that maximise the mutual information between predictions and model parameters, approximated via MC Dropout [2]:

$$\mathrm{BALD}(x) = H\big[\mathbb{E}_\theta[p(y|x,\theta)]\big] - \mathbb{E}_\theta\big[H[p(y|x,\theta)]\big]. \tag{4}$$

The first term is the entropy of the mean prediction; the second is the mean entropy across dropout masks, penalising aleatoric uncertainty.

Batch BALD [6]. Extends BALD from single-point selection to joint batch selection. Rather than ranking points independently by mutual information, Batch BALD scores candidate sets to reduce redundancy among highly uncertain examples.

D-Optimal Experimental Design. Greedily maximises $\det(\mathbf{X}^\top\mathbf{X})$ in embedding space, where $\mathbf{X}$ is the matrix of labeled embeddings. At each step the unlabeled point with the highest leverage score is chosen:

$$x^* = \arg\max_{x\in\mathcal{U}} \; f(x)^\top \big(\mathbf{X}^\top\mathbf{X} + \lambda I\big)^{-1} f(x), \tag{5}$$

with ridge $\lambda = 10^{-4}$. The inverse is maintained cheaply via Sherman-Morrison rank-1 updates. Because the criterion assumes a *fixed* embedding geometry, its effectiveness depends on the backbone being frozen (see Section 4.4).

3.6 Seed-Set Initialisation

Cold-start experiments compare alternative ways to construct the initial labelled seed set before the AL loop begins.

Random initialisation. Samples the seed set uniformly from the unlabeled pool.

CoreSet initialisation. Uses the same k-center criterion as CoreSet acquisition, but applies it once to maximise coverage of the embedding space at round $t = 0$.

TypiClust [3]. Designed for small-budget cold-start settings. The unlabeled pool is clustered into n clusters via Mini-Batch K-Means. Within each cluster the most *typical* point is selected, where typicality is defined as the inverse mean distance to the k-nearest neighbours ($k = 20$):

$$\mathrm{typ}(x) = \frac{1}{\bar{d}_k(x) + \epsilon}, \quad \bar{d}_k(x) = \frac{1}{k}\sum_{j=1}^{k} \|f(x) - f(x_j)\|_2. \tag{6}$$

Selecting typical high-density points is argued to be most useful when the label budget is extremely small.

D-Optimal initialisation. Applies the same leverage-score criterion as D-Optimal acquisition to construct an informative seed set before any model updates are made.

3.7 Transfer Pretraining

For transfer experiments, DNABERT-2 is first fine-tuned on a fully labelled source task before entering the AL loop on the target task. We study same-task cross-species transfer (mm→hs promoters). DNABERT-2 is trained on the complete mouse promoter training pool (no AL, supervised) for 10 epochs ($\mathrm{lr} = 2 \times 10^{-5}$, batch size 16). The resulting checkpoint serves as the pretrained backbone for the subsequent target-domain AL experiment. Two fine-tuning regimes are compared:

1. **Full fine-tuning**: all backbone weights are updated jointly with the classification head during each AL round.
2. **Frozen backbone**: backbone weights are frozen after pretraining; only the classification head is trained ($\mathrm{lr} = 10^{-3}$).

3.8 Evaluation Metric

Macro-averaged F1 (macro-F1) is reported throughout, providing equal weight to each class and appropriate for the balanced class distributions in all three datasets.

4 Results

4.1 Mouse Promoter Benchmark

We begin with the mouse promoter benchmark introduced in Section 3.2, using the default AL protocol from Section 3.3. This experiment compares Random, BALD, BADGE, and Batch BALD under a fixed 2,000-label budget.

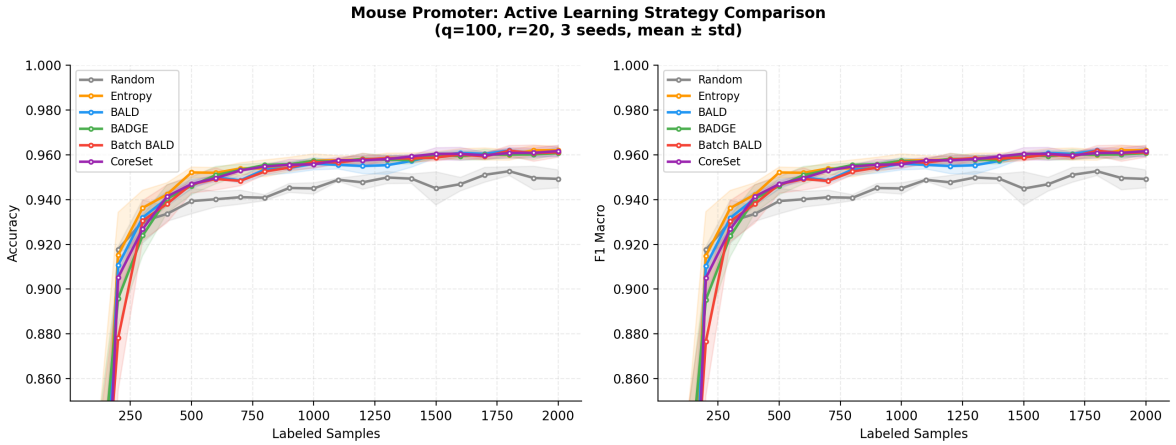

Figure 1: Learning curves (accuracy and F1 macro) for four active learning strategies on the mouse promoter task under fixed budget $q = 100$, $T = 20$. Shaded regions show ± 1 standard deviation over three random seeds.

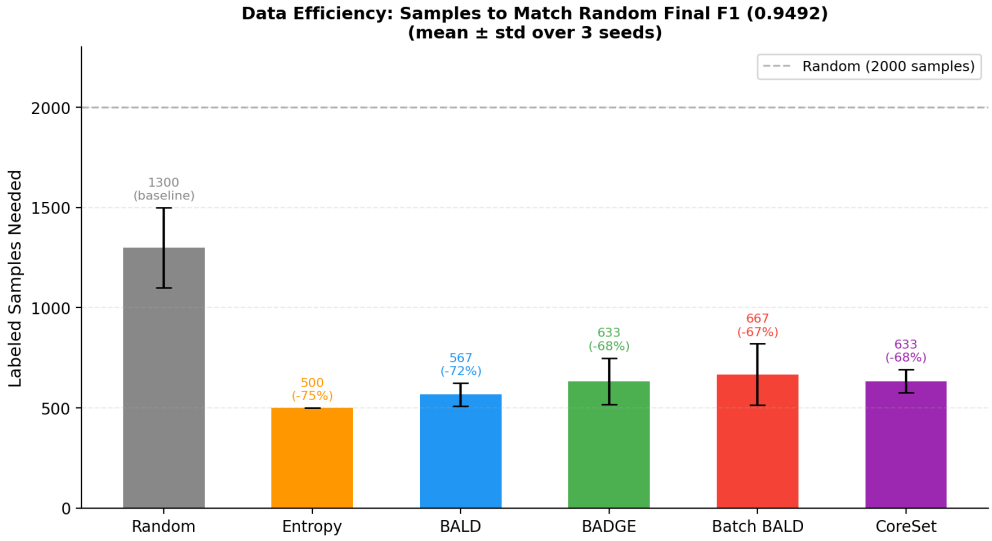

Figure 2: Data efficiency: number of labelled samples required by each strategy to match the random baseline's final F1 macro score. Bars below the dashed line indicate sample savings relative to random.

Figure 1 shows that BADGE and BALD improve label efficiency over random sampling in the low-budget regime, with BADGE providing the most consistent early advantage. By the full 2,000-label budget the gap narrows, indicating that the mouse promoter task can eventually be solved reasonably

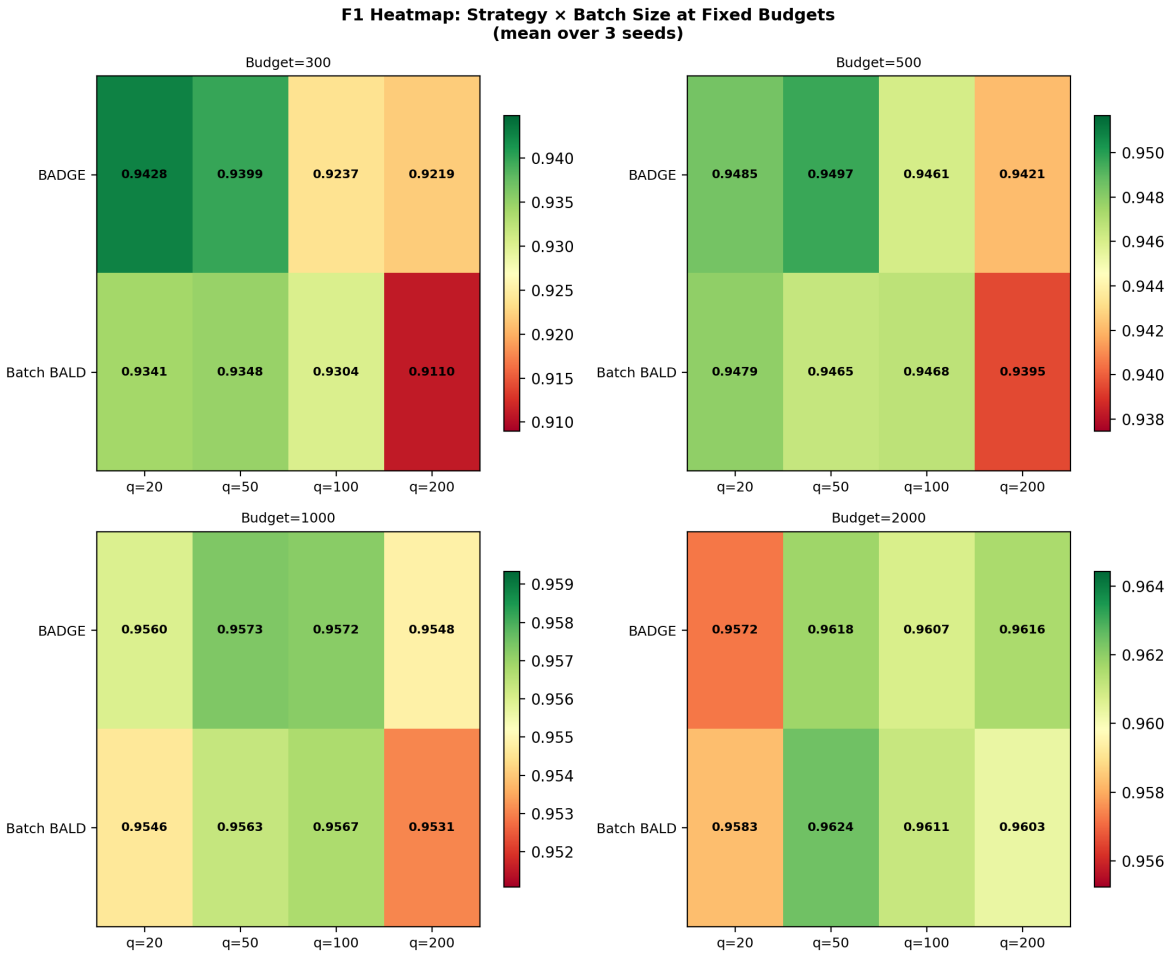

Figure 3: Strategy × batch size F1 heatmap (mean over 3 seeds) at four fixed budgets. BADGE is robust to batch size throughout; Batch BALD shows a larger drop at $q = 200$ under limited budgets.

well even by random querying, but active learning still reduces the number of labels needed to reach the same end performance.

This sample-efficiency advantage is quantified in Figure 2: BADGE requires the fewest labelled samples to match the final macro-F1 achieved by the random baseline, while Batch BALD remains competitive but less clearly separated. Under a fixed total budget, batch size has only a modest effect overall, but the interaction with strategy matters at limited budgets (Figure 3). At $n = 300$, BADGE is stable across batch sizes ($q = 20$: 0.943; $q = 200$: 0.922), whereas Batch BALD degrades more sharply with larger batches ($q = 20$: 0.934; $q = 200$: 0.910), consistent with its joint mutual-information approximation becoming less reliable when fewer AL rounds are available. We therefore treat batch size as a secondary design choice for BADGE but recommend small-to-moderate q for Batch BALD under tight label budgets. The final-score distributions across strategies and seeds are shown in Appendix Figure 9.

4.2 Cold-Start Initialisation

Table 2, Figure 4, and Appendix Figure 10 report macro-F1 across label budgets for three initialisation strategies. On mouse promoter detection, TypiClust yields the highest cold-start F1 (0.726 ± 0.039 at $n = 100$, vs. 0.661 for Random and 0.632 for CoreSet). The advantage is short-lived: from $n = 200$ onward TypiClust falls behind Random, and CoreSet achieves the highest F1 at $n = 2000$ (0.962 ± 0.007).

On NT enhancer detection the ordering reverses entirely. TypiClust is the weakest strategy at every evaluated budget, reaching only 0.541 ± 0.046 at $n = 100$ — below both Random (0.653) and

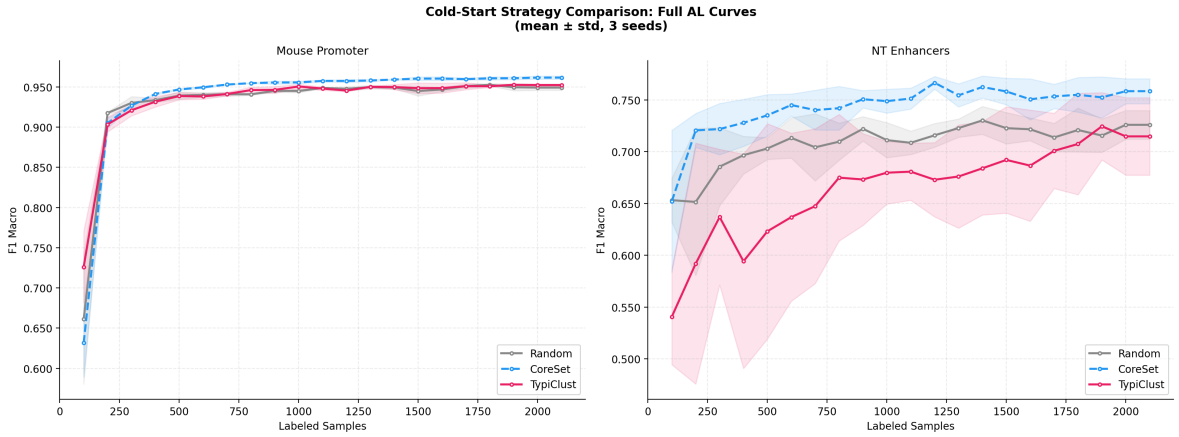

Figure 4: Full AL learning curves for Random, CoreSet, and TypiClust initialisation on mouse promoter (left) and NT enhancer (right) detection. Shaded bands show ±1 std across 3 seeds.

Table 2: Cold-start macro-F1 (mean ± std, 3 seeds). Bold: best per column.

Dataset	Strategy	$n = 100$	$n = 200$	$n = 300$	$n = 500$	$n = 2000$
Mouse Promoter	Random	0.661 ± 0.027	$\mathbf{0.918 \pm 0.001}$	$\mathbf{0.930 \pm 0.011}$	$\mathbf{0.939 \pm 0.009}$	0.949 ± 0.003
	CoreSet	0.632 ± 0.040	0.905 ± 0.014	0.927 ± 0.009	0.947 ± 0.003	$\mathbf{0.962 \pm 0.007}$
	TypiClust	$\mathbf{0.726 \pm 0.039}$	0.904 ± 0.016	0.921 ± 0.017	0.939 ± 0.011	0.952 ± 0.004
NT Enhancers	Random	$\mathbf{0.653 \pm 0.021}$	0.652 ± 0.071	0.686 ± 0.038	0.703 ± 0.011	0.726 ± 0.014
	CoreSet	0.652 ± 0.069	$\mathbf{0.721 \pm 0.016}$	$\mathbf{0.722 \pm 0.025}$	$\mathbf{0.735 \pm 0.020}$	$\mathbf{0.758 \pm 0.012}$
	TypiClust	0.541 ± 0.046	0.592 ± 0.116	0.637 ± 0.065	0.623 ± 0.104	0.715 ± 0.037

CoreSet (0.652) — and 0.715 ± 0.037 at $n = 2000$ compared to 0.758 ± 0.012 for CoreSet. CoreSet is the most consistent strategy overall, providing either the best or near-best F1 across all tasks and budgets.

4.3 Transfer Pretraining

Cross-species transfer (mm→hs). Transfer pretraining on mouse promoters yields a large, persistent advantage on human promoter detection (Figure 5, left; Table 3). At $n = 100$ the transferred model achieves 0.973 ± 0.003, compared to 0.750 ± 0.012 for the from-scratch baseline ($\Delta = +0.223$). The baseline does not close this gap within the evaluated range: at $n = 2000$ it reaches 0.959, still 0.014 below the transferred model.

4.4 D-Optimal Query Strategy

Figure 5 (right) and Table 3 isolate the effect of D-Optimal querying on the mm→hs task under two backbone regimes.

With full fine-tuning, D-Optimal provides no benefit over random AL: F1 is 0.974 for both at $n = 500$, and D-Optimal trails at $n = 2000$ (0.962 ± 0.016 vs. 0.966 ± 0.006, with markedly higher variance). The likely explanation is *embedding drift*: because the backbone weights are updated every round, the embedding geometry shifts after each fine-tuning step, making the D-optimal design computed on round t a poor approximation of the information matrix on round $t+1$.

With a frozen backbone, this instability disappears. The embedding space is fixed throughout the AL loop, so D-Optimal can exploit the true covariance structure of the unlabelled pool without the geometry shifting beneath it. D-Optimal with a frozen backbone is the best-performing condition at every evaluated budget, achieving 0.976 ± 0.000 at $n = 100$ and 0.973 ± 0.003 at $n = 2000$ with the lowest variance across seeds.

These results indicate that D-Optimal's advantage is contingent on two conditions being met simultaneously: a stable embedding geometry (frozen backbone) and an embedding that is already

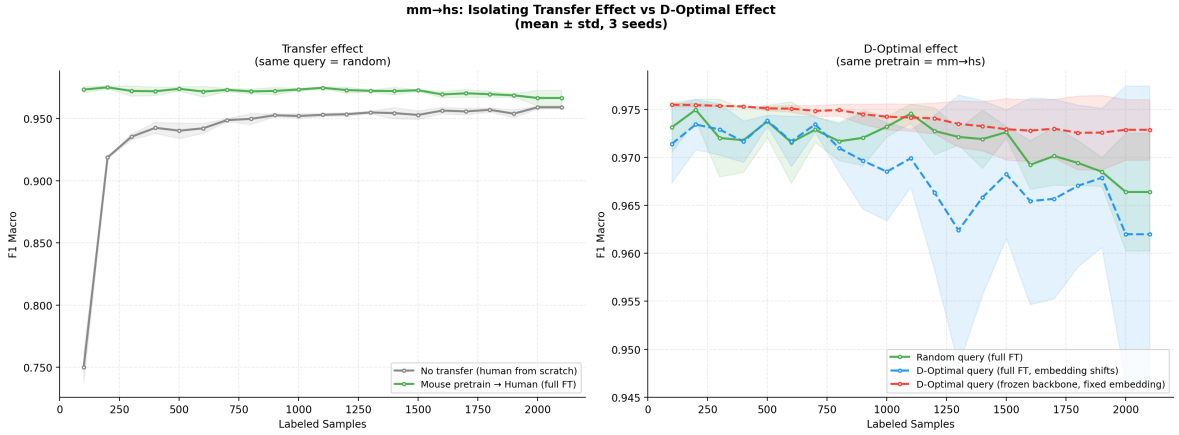

Figure 5: Isolation of transfer effect and D-Optimal effect on mm→hs promoter detection. *Left*: fixing query strategy to random, adding mouse pretraining raises F1 from 0.750 to 0.973 at $n = 100$. *Right*: fixing pretraining to mm→hs, D-Optimal with a frozen backbone (red dashed) consistently outperforms random AL (green), while D-Optimal with full fine-tuning (blue dashed) underperforms due to embedding drift across rounds.

Table 3: Effect of D-Optimal querying on mm→hs promoter (mean ± std, 3 seeds). Bold: best per column.

Condition	$n = 100$	$n = 500$	$n = 1000$	$n = 2000$
Baseline (no transfer, random)	0.750 ± 0.012	0.940 ± 0.006	0.952 ± 0.002	0.959 ± 0.001
Transfer, full FT, random AL	0.973 ± 0.003	0.974 ± 0.002	0.973 ± 0.001	0.966 ± 0.006
Transfer, full FT, D-Optimal AL	0.971 ± 0.004	0.974 ± 0.001	0.969 ± 0.005	0.962 ± 0.016
Transfer, frozen, D-Optimal AL	$\mathbf{0.976 \pm 0.000}$	$\mathbf{0.975 \pm 0.000}$	$\mathbf{0.974 \pm 0.001}$	$\mathbf{0.973 \pm 0.003}$

discriminative for the target task (provided here by cross-species transfer pretraining).

4.5 Fungi Multi-Class Classification

We next evaluate a harder 20-class benchmark from GUE using the same DNABERT-2 backbone and the acquisition functions defined in Section 3.5. Unless otherwise stated, the protocol is the same as in Section 3.3; the only additional study varies the query batch size while keeping the total label budget fixed.

BADGE with q=50 per round achieves the highest final F1 macro score of 0.454±0.003 and the best accuracy (0.460), placing all four BADGE configurations in the top-4 positions (Figure 6). CoreSet and Random reach competitive scores (∼0.43 F1), both outperforming all BALD variants on this task. Standard BALD ranks 11th out of 13 configurations (0.412 F1), and D-Optimal performs worst at 0.346 F1, more than 10 percentage points below BADGE.

The same ranking is reflected in the data-efficiency and final-score comparisons in Appendix Figures 13 and 14. Under a fixed 2,000-label budget, batch size has only a secondary effect: the moderate configuration q=50, r=40 is the strongest BADGE setting, but the spread across $q \in \{20, 50, 200\}$ is much smaller than the gap between the diversity-based methods and the uncertainty-based methods (Appendix Figure 12).

These results reveal that for high-dimensional multi-class genomic classification, diversity-based sampling (BADGE, CoreSet) outperforms uncertainty-based methods (BALD, Batch BALD), and classical experimental design (D-Optimal) is ill-suited for deep-learning active learning in this setting.

4.6 Initialisation vs. Query Strategy

A core practical question in pool-based active learning is whether the *initialisation* of the labelled seed set matters as much as the choice of *query strategy* applied in subsequent rounds. To answer this we

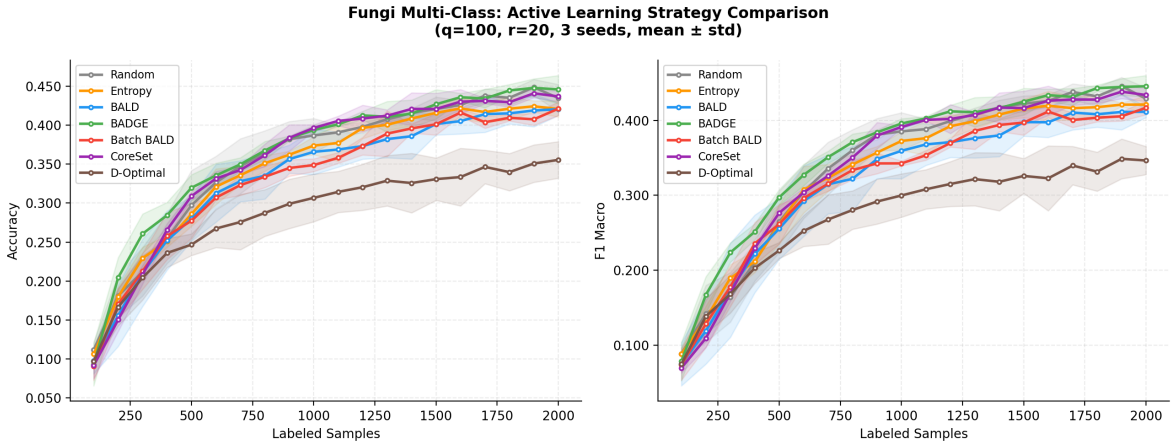

Figure 6: Learning curves for seven active learning strategies on the 20-class fungi classification task. Each curve shows the mean over three seeds; shaded bands show ±1 std.

designed a 2×3 factorial experiment on the mouse promoter binary classification task, crossing two initialisation methods with three query strategies.

Factors.

- **Initialisation strategy** (2 levels): *Random* (uniform random seed set) vs. *D-optimal* (initial pool selected via D-optimality criterion in DNABERT-2 representation space).
- **Query strategy** (3 levels): *Random*, *BADGE*, and *D-optimal querying*.

In addition, an initial pool size (n_{init}) sensitivity sweep tests $n_{\text{init}} \in \{100, 300, 500\}$ for the D-optimal init + BADGE query combination. Each of the 24 experimental conditions is replicated with seeds 42, 43, and 44. The fixed budget is $n_0 = 100$, $q = 100$, $T = 20$ (2,000 total labels). Twenty-four WandB runs were collected and aggregated into 480 round-level observations.

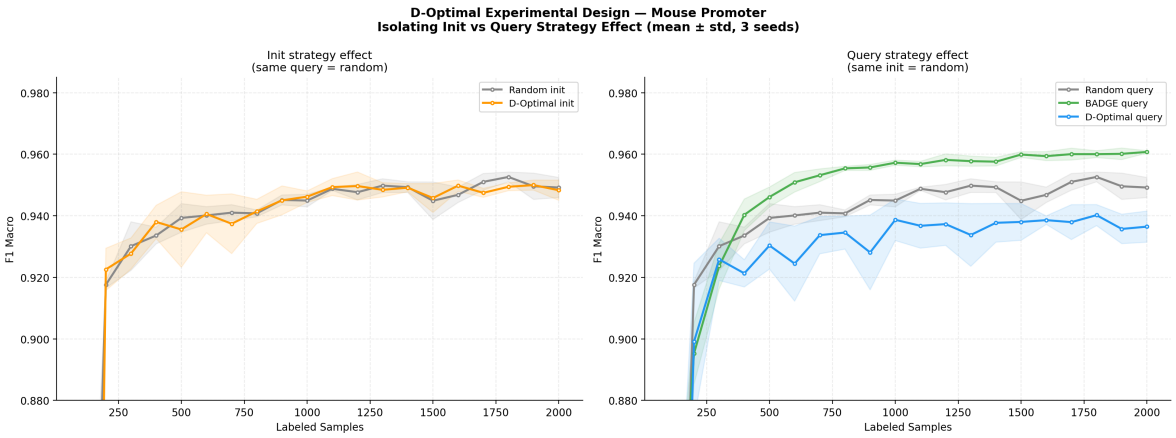

Figure 7: Factorial effect plot: learning curves separated by initialisation method (columns) and query strategy (rows) on the mouse promoter task. Solid lines indicate random initialisation; dashed lines indicate D-optimal initialisation.

The results reveal a clear main effect of *query strategy*, with BADGE querying consistently achieving the highest final F1 macro (≈ 0.96) regardless of the initialisation method used (Figure 7). D-optimal querying significantly underperforms (0.936–0.937 F1), even when paired with D-optimal initialisation.

Initialisation strategy has a negligible main effect: D-optimal initialisation with random querying (0.948 F1) is virtually indistinguishable from random initialisation with random querying (0.949 F1),

suggesting that cold-start optimisation provides minimal benefit when the subsequent query strategy is simple. Appendix Figure 15 shows the same ordering across all six conditions.

The initial pool size sweep in Appendix Figure 16 shows that $n_{\text{init}} = 300$ yields the strongest early performance (0.928 F1 at 200 total labels), but all pool sizes converge to comparable final performance (≈ 0.96 F1) by 2,000 labels, indicating diminishing returns for larger initial investments.

Takeaway. Query strategy dominates initialisation strategy for this binary DNA promoter classification task. Practitioners should prioritise a high-quality query function (BADGE) over investing effort in optimised cold-start initialisation.

5 Discussion and Conclusion

The experiments show that active learning can improve annotation efficiency for genomic sequence classification, but the gains are not uniform across strategies or tasks. On the promoter and fungi benchmarks, diversity-based methods are the most dependable, with BADGE emerging as the strongest overall choice and CoreSet remaining a robust alternative. In contrast, uncertainty-only methods are less stable, and their advantage largely disappears on the harder multi-class setting.

Two secondary design choices appear less important than the acquisition rule itself. Cold-start optimisation helps only in the first few rounds and does not reliably change the final ordering, while query batch size has only a modest effect under a fixed total labelling budget. The more substantive interaction is between transfer and representation geometry: same-task cross-species transfer is strongly beneficial, and D-Optimal querying succeeds specifically in the frozen, well-aligned mm→hs setting where the embedding geometry is stable and task relevant.

Why Does BADGE Perform Consistently Well?

BADGE [1] clusters gradient embeddings in the final-layer parameter space, jointly capturing uncertainty (gradient magnitude peaks near the decision boundary) and diversity (k-means++ spread suppresses redundant queries). Both signals are well-matched to DNABERT-2: the pretrained backbone produces geometry-preserving embeddings, so gradient directions faithfully encode class uncertainty, and the iterative k-means++ selection naturally adapts to the representation shifts that occur during fine-tuning. This round-by-round adaptability — which static methods such as D-Optimal lack — explains BADGE's consistent top-rank across all three benchmarks.

Why Does TypiClust Help Only at Small Budgets?

TypiClust [3] selects cluster prototypes from the embedding space, guaranteeing that the initial label set covers the major input modes regardless of random seed. This directly addresses the dominant failure mode of small-budget random initialisation — missing an entire class — and explains the mouse promoter gain at $n = 100$ (0.726 ± 0.039 vs. 0.661 for Random, with lower variance). Once a modest labelled set exists, however, the most informative quer ies lie near the decision boundary rather than at dense cluster centres. TypiClust's coverage criterion systematically avoids these boundary samples, so adaptive methods such as BADGE and CoreSet take over from $n = 200$ onward. The NT enhancer results further show that TypiClust provides no benefit when the embedding clusters do not align with the class structure, limiting its utility to tasks with well-separated representations at very small budgets.

When Does D-Optimal Design Work?

D-Optimal design maximises $\det(\mathbf{X}^\top \mathbf{X})$ under a fixed feature map $f(\cdot)$. When the backbone is jointly fine-tuned, this assumption is violated at every round: the embedding geometry shifts after each model update, so the selected points may be suboptimal by the next round, and greedy iterative application accumulates this mismatch over the AL loop. The result is no advantage over random querying under full fine-tuning (Table 3).

These limitations disappear in the frozen-backbone setting. After mm→hs transfer pretraining, the embedding geometry is fixed and already discriminative for the target task, so the linear assumption

holds by construction. D-Optimal achieves the best performance at every evaluated budget with the lowest variance across seeds (Table 3) — the expected signature of a well-specified design criterion in a stable function space. In summary, D-Optimal is well suited to transfer-then-freeze workflows but not to settings where the backbone is being adapted jointly with the AL loop.

Overall, the report suggests a pragmatic recipe for similar problems: start with a strong pretrained encoder, use BADGE or CoreSet as the default acquisition rule, treat batch size as a secondary tuning knob, and rely on D-Optimal design only when the embedding space is known to be stable and relevant to the target task.

References

[1] Jordan T. Ash, Chicheng Zhang, Akshay Krishnamurthy, John Langford, and Alekh Agarwal. Deep batch active learning by diverse, uncertain gradient lower bounds, 2020.

[2] Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning, 2016.

[3] Guy Hacohen, Avihu Dekel, and Daphna Weinshall. Active learning on a budget: Opposite strategies suit high and low budgets, 2022.

[4] Neil Houlsby, Ferenc Huszár, Zoubin Ghahramani, and Máté Lengyel. Bayesian active learning for classification and preference learning, 2011.

[5] Yanrong Ji, Zhihan Zhou, Han Liu, and Ramana V Davuluri. Dnabert: pre-trained bidirectional encoder representations from transformers model for dna-language in genome. *Bioinformatics*, 37(15):2112–2120, 02 2021.

[6] Andreas Kirsch, Joost van Amersfoort, and Yarin Gal. Batchbald: Efficient and diverse batch acquisition for deep bayesian active learning, 2019.

[7] Ozan Sener and Silvio Savarese. Active learning for convolutional neural networks: A core-set approach, 2018.

[8] Zhihan Zhou, Yanrong Ji, Weijian Li, Pratik Dutta, Ramana Davuluri, and Han Liu. Dnabert-2: Efficient foundation model and benchmark for multi-species genome, 2024.

A Supplementary Figures

A.1 Mouse Promoter

Figures 8 and 9 provide supplementary detail on the mouse promoter experiments. Figure 8 shows learning curves for varying query batch size under a fixed total label budget. Figure 9 summarises the distribution of final F1 and accuracy across strategies and seeds.

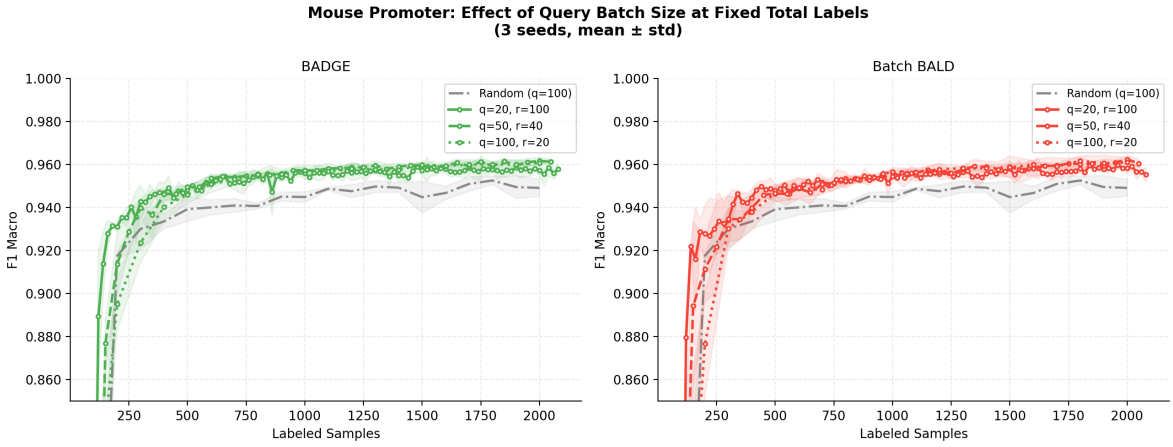

Figure 8: Effect of query batch size on BADGE and Batch BALD performance on mouse promoter data, keeping total label budget fixed at 2,000 samples.

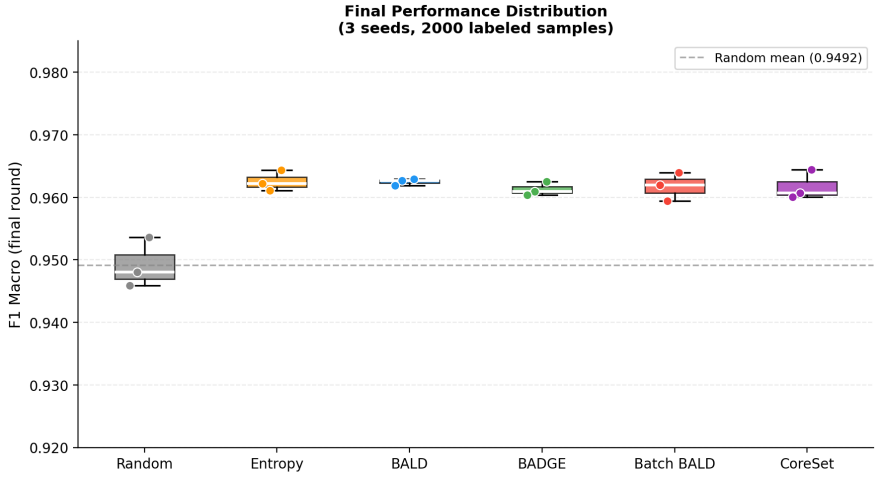

Figure 9: Boxplots of final F1 macro and accuracy for all strategies on the mouse promoter task. Individual seed results are overlaid as scatter points.

A.2 Cold-Start and Transfer

Figure 10 zooms into the low-budget regime of the cold-start experiment. Figure 11 isolates the transfer and D-Optimal effects on the mm→hs task, comparing full fine-tuning and frozen backbone conditions.

A.3 Fungi

Figures 12–14 complement the main fungi results. Figure 12 examines batch size sensitivity; Figure 13 reports data efficiency relative to the random baseline; Figure 14 shows final performance distributions across all strategies.

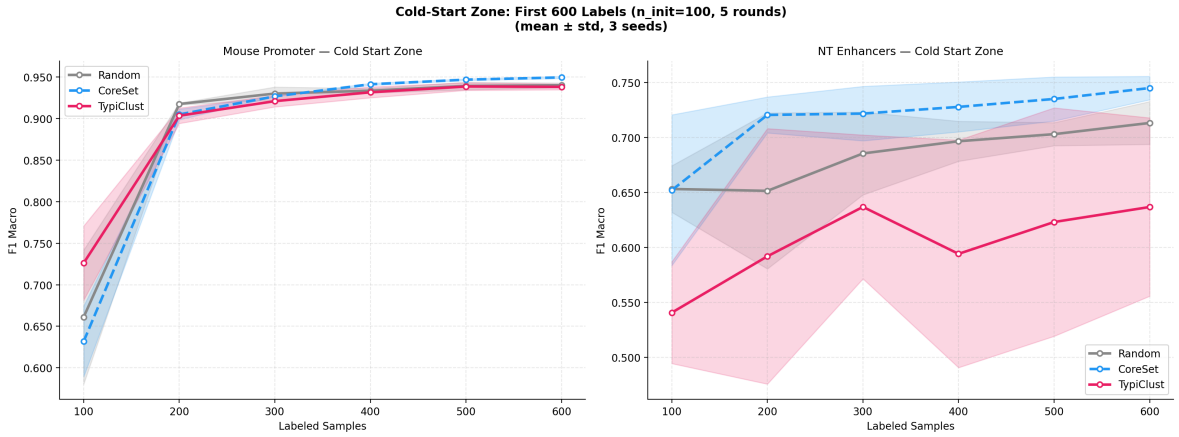

Figure 10: Cold-start zone ($n \leq 600$) detail. TypiClust leads at $n = 100$ on the promoter task but falls behind from $n = 200$ onward; on NT enhancers it is the weakest strategy throughout.

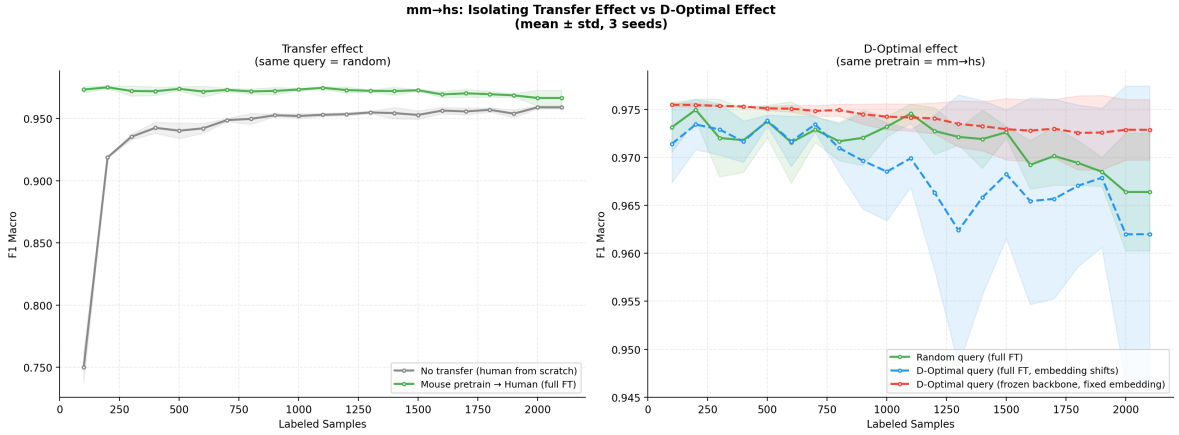

Figure 11: Isolation of transfer effect (left) and D-Optimal effect (right) on the mm→hs task. With a frozen backbone, D-Optimal is the best-performing condition at all label budgets. With full fine-tuning, D-Optimal offers no advantage over random querying.

A.4 Factorial Study

Figures 15 and 16 provide additional detail on the D-Optimal factorial study. Figure 15 shows full learning curves and an early-budget bar chart for all six init×query combinations. Figure 16 examines sensitivity to the initial pool size n_{init}.

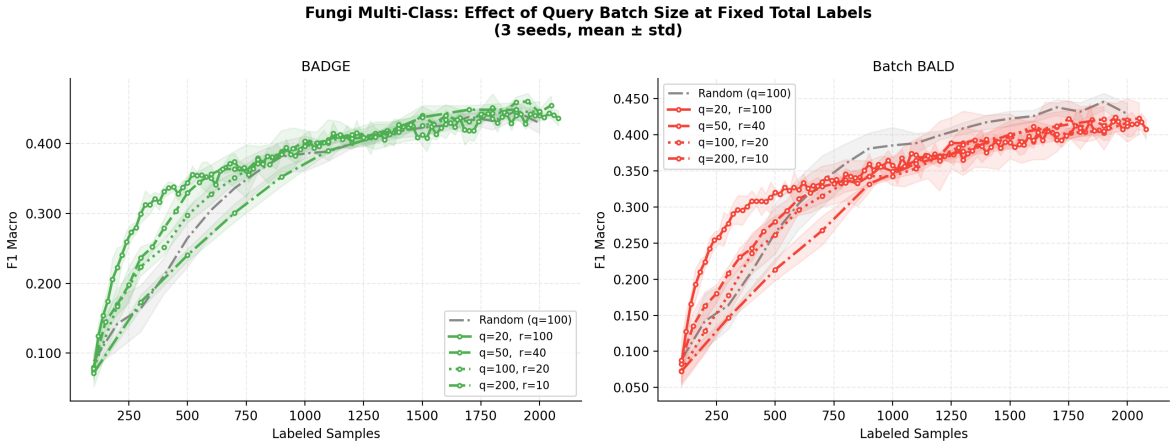

Figure 12: Query batch size effect ($q \in \{20, 50, 200\}$) on BADGE and Batch BALD performance for the fungi multi-class task, total budget fixed at 2,000 labels.

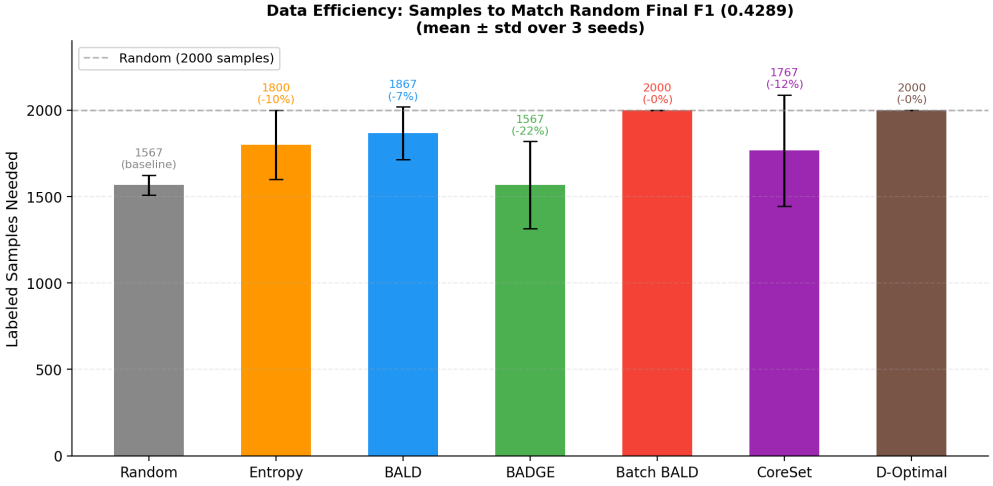

Figure 13: Data efficiency on the fungi task: labelled samples needed to reach the random baseline's final F1 macro score.

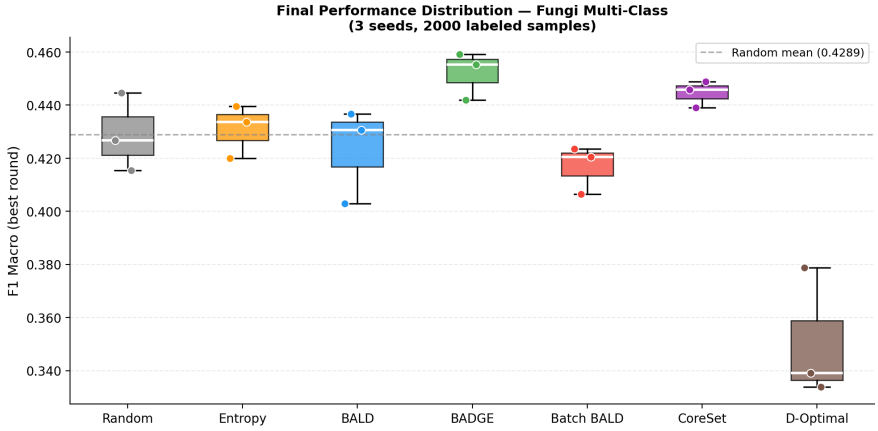

Figure 14: Final performance distributions (F1 macro, accuracy) for all strategies on the fungi task. Boxplots with individual seed scatter.

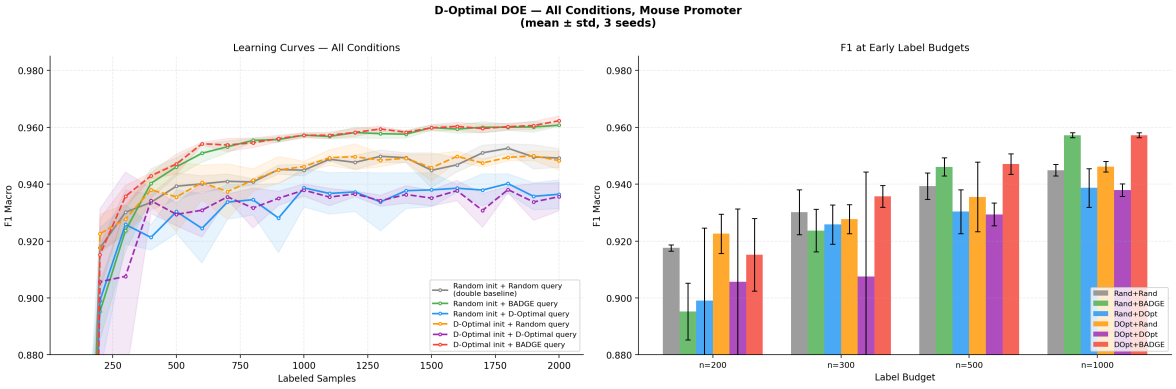

Figure 15: All six init×query conditions shown as full learning curves (left) and early-budget bar chart (right) at label counts 200, 300, 500, and 1000.

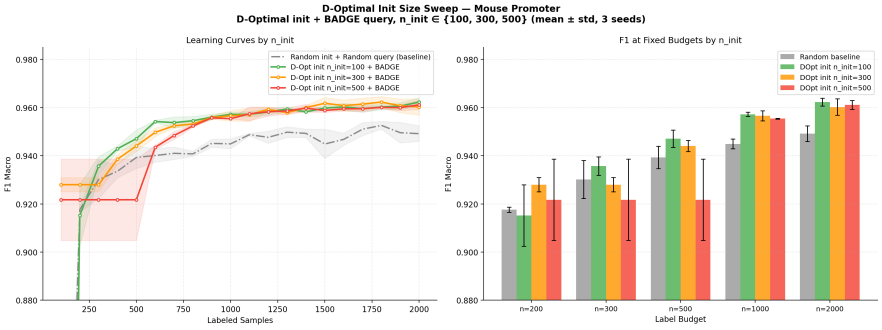

Figure 16: Sensitivity of D-Optimal init + BADGE query to initial pool size $n_{\text{init}} \in \{100, 300, 500\}$. Learning curves and a bar comparison at $n = 200$ labels.